Source Code :

news.py - D:\semester\4.1\New folder\pro\news_project 2nd\news_project\news.py (3.14.5)

File Edit Format Run Options Window Help

```
"""Step 1: collect news articles for a topic (no API key needed)."""
import re
import feedparser
try:
    import trafilatura
except Exception:       # blocked or missing: fall back to snippets
    trafilatura = None
from urllib.parse import urlparse, parse_qs, quote_plus
from concurrent.futures import ThreadPoolExecutor


def _real_url(link):
    # Bing wraps the real article link inside ?url=...
    q = parse_qs(urlparse(link).query)
    return q.get("url", [link])[0]


def _clean_html(text):
    return re.sub(r"<[^>]+>", " ", text or "").strip()


def _get_full_text(url):
    if trafilatura is None:
        return ""
    try:
        html = trafilatura.fetch_url(url)
        if html:
            return trafilatura.extract(html) or ""
    except Exception:
        pass
    return ""


def fetch_articles(query, limit=15):
    feed = feedparser.parse(
        f"https://www.bing.com/news/search?q={quote_plus(query)}&format=rss"
    )
    items, seen = [], set()
    for e in feed.entries:
        title = e.get("title", "").strip()
        if not title or title in seen:
            continue
        seen.add(title)
        items.append({
            "title": title,
            "url": _real_url(e.get("link", "")),
            "snippet": _clean_html(e.get("summary", "")),
            "source": e.get("news_source", "News"),
            "published": e.get("published", ""),
        })
        if len(items) >= limit:
            break

    # download full article text in parallel (faster)
    with ThreadPoolExecutor(8) as pool:
        texts = list(pool.map(lambda a: _get_full_text(a["url"]), items))

    for a, t in zip(items, texts):
        # if the full text could not be downloaded, use title + snippet
        a["text"] = t if len(t) > 300 else f'{a["title"]}. {a["snippet"]}'
    return items
```

Ln: 1 Col: 0

nlp.py - D:\semester\4.1\New folder\pro\news_project 2nd\news_project\nlp.py (3.14.5)

File Edit Format Run Options Window Help

```
"""Summarization + clustering in plain Python (no scikit-learn, no numpy)."""
import re
import math
import random
from collections import Counter

STOP = set("""the and for that with this from are was were has have had not but you your they their them his her its
will would can could should may might also into than then there here about after before over under more most some such
only other which when where while what who whom why how all any both each few many much own same too very just said says
say new one two been being out off our ours she him who's per via get got use used""".split())


def tokenize(text):
    return [w for w in re.findall(r"[a-z]{3,}", text.lower()) if w not in STOP]


def tfidf_vectors(docs, normalize=True):
    """Turn each text into a {word: weight} dictionary using TF-IDF."""
    tokens = [tokenize(d) for d in docs]
    n = len(docs)
    df = Counter()
    for t in tokens:
        df.update(set(t))
    idf = {w: math.log((1 + n) / (1 + c)) + 1 for w, c in df.items()}
    vectors = []
    for t in tokens:
        v = {w: c * idf[w] for w, c in Counter(t).items()}
        if normalize:
            norm = math.sqrt(sum(x * x for x in v.values())) or 1.0
            v = {w: x / norm for w, x in v.items()}
        vectors.append(v)
    return vectors


def cosine(a, b):
    if len(a) > len(b):
        a, b = b, a
    return sum(x * b.get(w, 0.0) for w, x in a.items())


# ---------------- Summarization ----------------
def split_sentences(text):
    parts = re.split(r"(?<=[.!?])\s+", text.replace("\n", " "))
    return [p.strip() for p in parts if 40 <= len(p.strip()) <= 400]


def summarize(text, n=3):
    """Pick the n most important sentences (TF-IDF score)."""
    sents = split_sentences(text)
    if not sents:
        return text[:300]
    if len(sents) <= n:
        return " ".join(sents)
    vectors = tfidf_vectors(sents, normalize=False)
    scores = [sum(v.values()) / math.sqrt(max(len(v), 1)) for v in vectors]
    best = sorted(sorted(range(len(sents)), key=lambda i: scores[i])[-n:])
    return " ".join(sents[i] for i in best)
```

app.py - D:\semester\4.1\New folder\pro\news_project 2nd\news_project\app.py (3.14.5)

File Edit Format Run Options Window Help

```python
"""Step 4: the website."""
import time
import threading
import webbrowser
from flask import Flask, render_template, abort, request
from news import fetch_articles
from nlp import summarize, cluster_articles

app = Flask(__name__)

TOPICS = {
    "t20":     {"title": "T20 Cricket World Cup", "query": "T20 Cricket World Cup", "icon": "🏏"},
    "trade":   {"title": "American Trade Wars",   "query": "US tariffs trade war",  "icon": "⚖️"},
    "quantum": {"title": "Quantum Computing",     "query": "quantum computing",     "icon": "⚛️"},
    "space":   {"title": "Space News",            "query": "ISRO NASA space mission", "icon": "🚀"},
    "nobel":   {"title": "Nobel Prizes",          "query": "Nobel Prize",           "icon": "🏅"},
}

CACHE = {}            # remembers results so pages open fast
CACHE_SECONDS = 1800  # 30 minutes


def build_topic(key):
    articles = fetch_articles(TOPICS[key]["query"])
    for a in articles:
        a["summary"] = summarize(a["text"], n=2)
    overall = summarize(" ".join(a["summary"] for a in articles), n=4) if articles else ""
    return {"articles": articles, "overall": overall,
            "clusters": cluster_articles(articles), "time": time.time()}


@app.route("/")
def index():
    return render_template("index.html", topics=TOPICS)


@app.route("/topic/<key>")
def topic(key):
    if key not in TOPICS:
        abort(404)
    cached = CACHE.get(key)
    if request.args.get("refresh") or not cached or time.time() - cached["time"] > CACHE_SECONDS:
        cached = CACHE[key] = build_topic(key)
    return render_template("topic.html", t=TOPICS[key], key=key, data=cached)


if __name__ == "__main__":
    url = "http://127.0.0.1:5000"
    # open the browser 1 second after the server starts
    threading.Timer(1.0, lambda: webbrowser.open(url)).start()
    app.run(debug=False, port=5000)
```

Ln: 52 Col: 0

```python
# ---------------- Clustering ----------------
def _centroid(members):
    total = Counter()
    for v in members:
        for w, x in v.items():
            total[w] += x
    norm = math.sqrt(sum(x * x for x in total.values())) or 1.0
    return {w: x / norm for w, x in total.items()}


def _kmeans(vectors, k, iterations=20):
    random.seed(42)
    centers = [random.choice(vectors)]
    while len(centers) < k:                          # pick far-apart starting points
        far = min(vectors, key=lambda v: max(cosine(v, c) for c in centers))
        centers.append(far)
    labels = [0] * len(vectors)
    for _ in range(iterations):
        new = [max(range(k), key=lambda c: cosine(v, centers[c])) for v in vectors]
        if new == labels:
            break
        labels = new
        for c in range(k):
            members = [vectors[i] for i in range(len(vectors)) if labels[i] == c]
            if members:
                centers[c] = _centroid(members)
    return labels, centers


def cluster_articles(articles, k=3):
    if len(articles) < 4:
        return [{"label": "All articles", "articles": articles}]
    texts = [a["title"] + " " + a["text"] for a in articles]
    vectors = tfidf_vectors(texts)
    k = max(2, min(k, len(texts) // 2))
    labels, centers = _kmeans(vectors, k)

    groups = []
    for c in range(k):
        members = [articles[i] for i in range(len(articles)) if labels[i] == c]
        if not members:
            continue
        top = sorted(centers[c], key=centers[c].get, reverse=True)[:3]
        groups.append({"label": ", ".join(top), "articles": members})
    return sorted(groups, key=lambda g: -len(g["articles"]))
```

Input and Output:

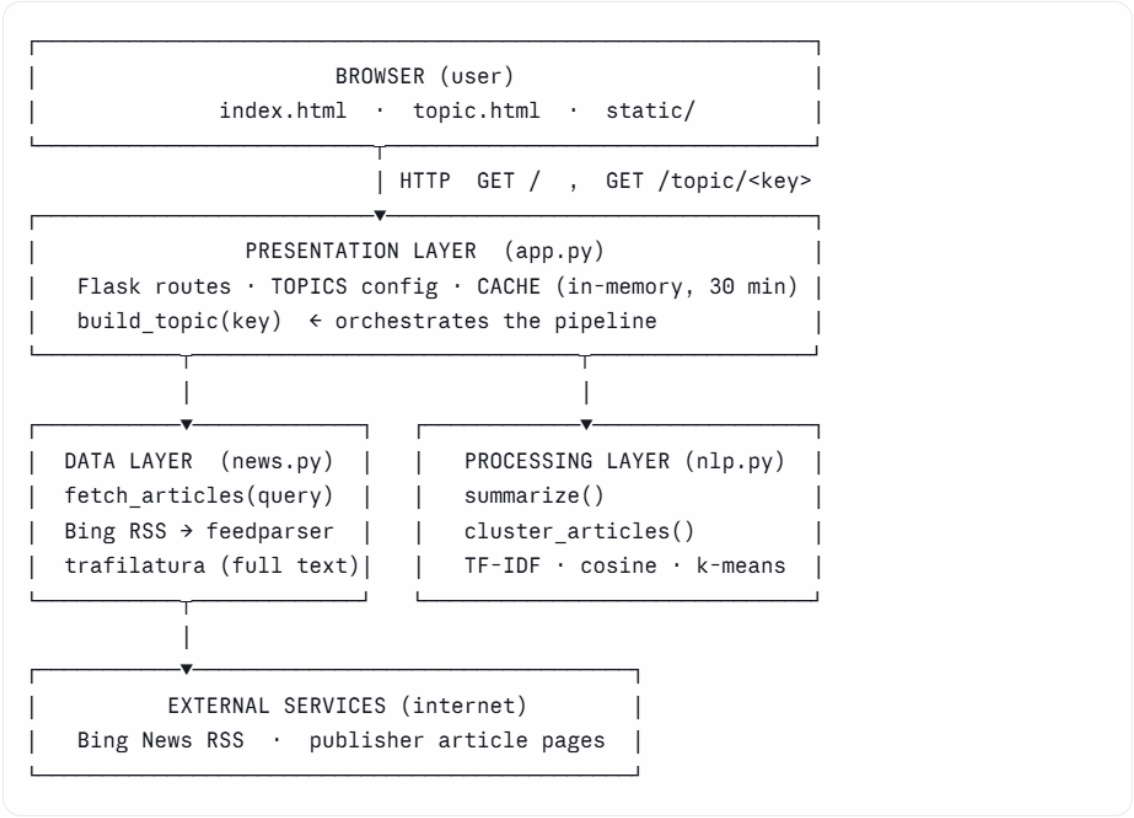
Input: The user selects one of five news topics (T20 Cricket World Cup, American Trade Wars, Quantum Computing, Space News, or Nobel Prizes) by clicking its card on the home page. The user does not type anything. Behind the scenes, the web uses a preset search phrase for the chosen topic and collects live news articles for it from Bing News RSS feeds.

Output: A web page for that topic showing:

- an overall summary of the topic in 4 sentences,
- a 2-sentence summary of every article,
- the articles grouped into themes, each labeled with its top keywords,
- the title, source, date and a link to each original article,
- the time the page was last updated.

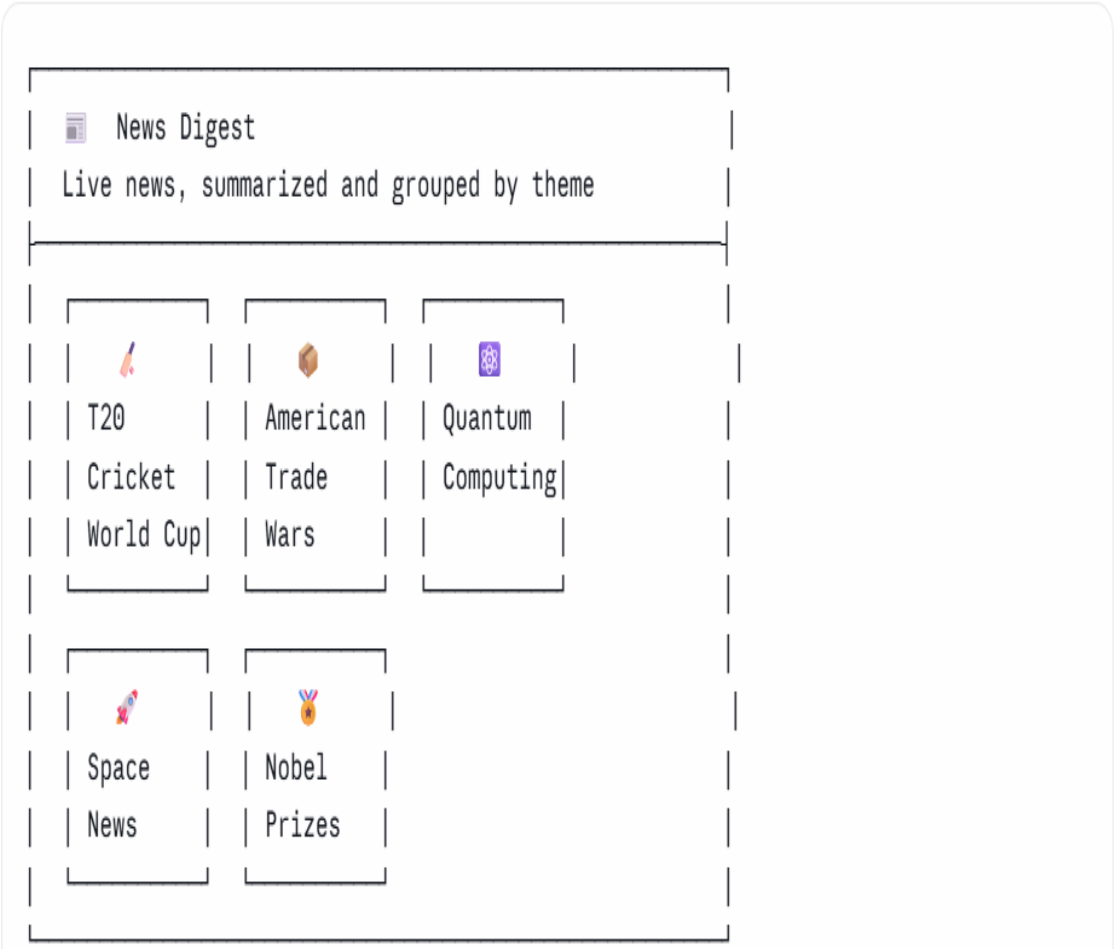
Architecture:

High-level view

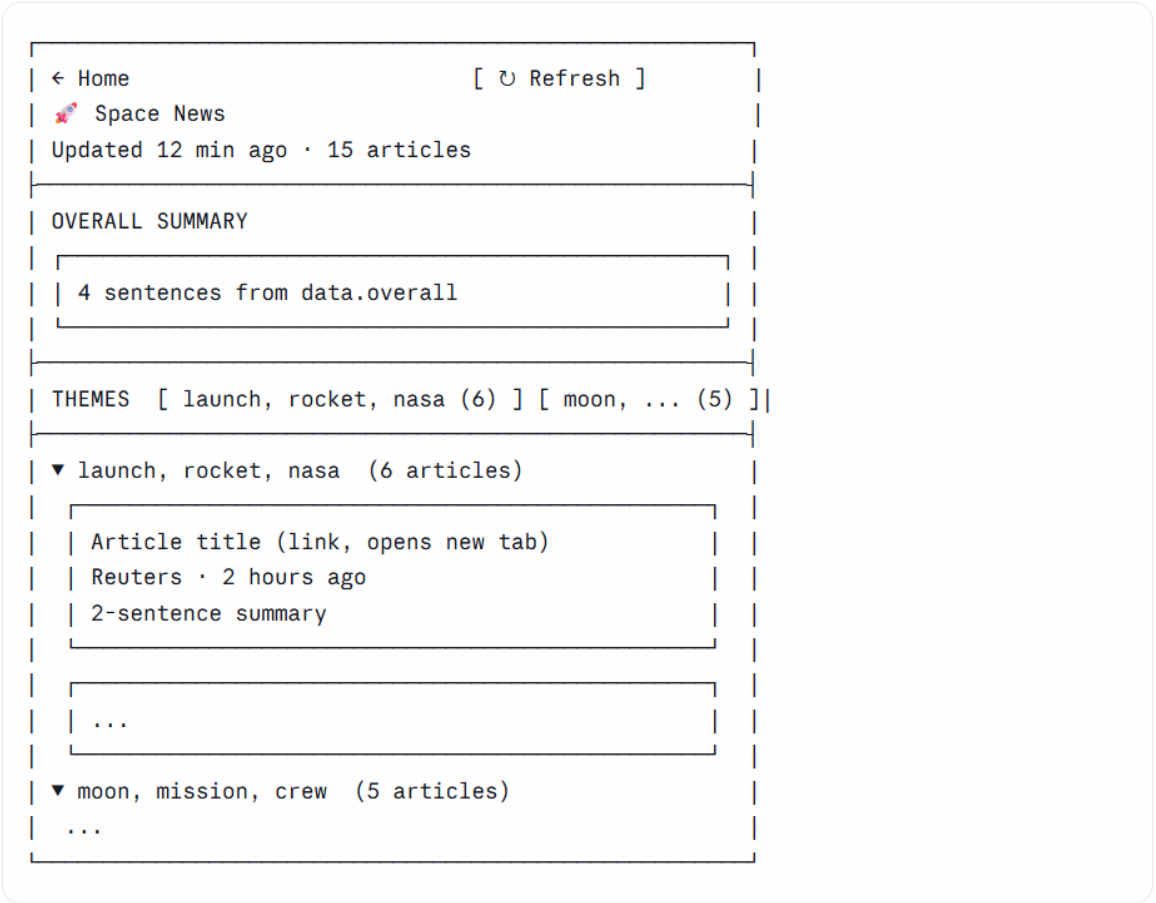


UI design:

Home page (`index.html`)



Topic page (`topic.html`)



1.Presentation:

Live news, summarized and grouped by theme

A Flask web application with automatic summarization and topic clustering

Python | Flask | TF-IDF | K-means

1. Purpose

- **Problem:** Reading many news articles on one subject takes a lot of time.
- **Goal:** Collect live articles, shorten each one, and sort them into themes automatically.
- **Result:** Give readers the whole story of a topic on a single page, with links to the originals.

- **Simple by design:** No API key, no account, and no heavy machine-learning libraries.

2. Input

The user types nothing. The only input is choosing a topic.

Data source: live Bing News RSS feed, plus the full text of each article (up to 15 per topic).

Topic card	Search phrase used behind the scenes
T20 Cricket World Cup	T20 Cricket World Cup
American Trade Wars	US tariffs trade war
Quantum Computing	quantum computing
Space News	ISRO NASA space mission
Nobel Prizes	Nobel Prize

3. How It Works

Step	What happens
1. Collect (news.py)	Reads the RSS feed, removes duplicates, downloads full article text in parallel
2. Summarize (nlp.py)	Scores sentences with TF-IDF and keeps the 2 most important per article
3. Overall summary	Summarizes the article summaries into 4 sentences
4. Cluster (nlp.py)	K-means groups articles into about 3 themes, labeled with top keywords
5. Present (app.py)	Flask renders the results as a web page and caches them for 30 minutes

4. Output

A web page for the chosen topic that contains:

- A 4-sentence overall summary of the topic
- Theme chips: keyword labels with article counts
- Articles grouped by theme, each with title, source, date and a 2-sentence summary
- A link to every original article
- An "Updated X min ago" line and a Refresh button

The output changes as new articles are published.

5. Conclusion

- News Digest turns a long list of articles into a short, organized overview of a topic.
- It shows that useful text summarization and clustering can be done with plain Python and no paid services.
- The design keeps data collection, language processing and the website separate, so each part can be upgraded on its own.

Demo Video Link: https://drive.google.com/file/d/17T5kU-etBINPSIktzUttZP4z2LNgvUdP/view?usp=sharing](https://drive.google.com/file/d/17T5kU-etBINPSIktzUttZP4z2LNgvUdP/view?usp=sharing)

Github Link: https://github.com/abhisheksree/news-project